

Patient-Level Prediction of 30-Day Hospital Readmission in Diabetic Inpatients

Research methodology notebook accompanying the paper *"Beyond Encounter-Level Benchmarks: Patient-Level Validation, Imbalance Corrections and Feature-Block Contributions in 30-Day Readmission Prediction"*

1. Research context

Unplanned readmission within 30 days of discharge is a headline quality and cost indicator: under the U.S. Hospital Readmissions Reduction Program (HRRP) hospitals are financially penalised for excess readmissions, and diabetes is among the conditions with the highest readmission burden. Risk-prediction models are meant to target scarce transitional-care resources at the patients most likely to bounce back.

Three methodological problems recur in the published literature on this benchmark:

1. **Encounter-level splitting.** The `Diabetes 130-US hospitals` dataset contains 101,766 *encounters* from only 71,518 *patients*. Splitting at random puts different admissions of the same patient in train and test, leaking patient-specific information. We quantify the resulting optimism.
2. **Reflexive imbalance "correction".** Positives are ~11%. SMOTE / undersampling are applied almost by default, and evaluated with threshold metrics such as accuracy or F1. We evaluate their effect on *ranking* (AUPRC), *calibration* (Brier, ECE) and *clinical utility* (net benefit).
3. **Unclear provenance of signal.** We run a feature-block ablation to separate demographic, administrative (non-clinical), clinical and prior-utilisation contributions.

2. Objectives

- **O1** Build a reproducible patient-level pipeline for 30-day readmission prediction.
- **O2** Quantify optimism induced by encounter-level versus patient-level splitting.

- **O3** Compare regularised logistic regression with random forests and gradient boosting under identical preprocessing and tuning budgets.
- **O4** Test whether imbalance corrections improve discrimination or utility.
- **O5** Identify which feature blocks and which individual predictors drive risk.

```
In [1]: import json, os, time, warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
from scipy import stats

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)

ROOT = Path("/home/user/RM_Lab")
DATA = ROOT / "data" / "diabetic_data.csv"
FIG = ROOT / "paper" / "figures"
TAB = ROOT / "paper" / "tables"
for p in (FIG, TAB):
    p.mkdir(parents=True, exist_ok=True)

# ---- Figure style: print-first, colour-blind-safe, validated categorical p
C = {"blue": "#2a78d6", "orange": "#eb6834", "aqua": "#1baf7a", "violet": "#
    "ink": "#0b0b0b", "ink2": "#52514e", "muted": "#8a8984", "grid": "#e4e3
    "pos": "#eb6834", "neg": "#2a78d6"}
SERIES = [C["blue"], C["orange"], C["aqua"], C["violet"]]
DASH = [(None, None), (5, 2), (1.6, 1.6), (7, 2, 1.6, 2)]

mpl.rcParams.update({
    "figure.dpi": 130, "savefig.dpi": 300, "savefig.bbox": "tight",
    "font.family": "serif", "font.serif": ["DejaVu Serif"], "font.size": 9,
    "axes.titlesize": 9.5, "axes.labelsize": 9, "legend.fontsize": 8,
    "xtick.labelsize": 8, "ytick.labelsize": 8,
    "axes.edgecolor": C["ink2"], "axes.linewidth": 0.7,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": C["grid"], "grid.linewidth": 0.6,
    "axes.axisbelow": True, "legend.frameon": False, "lines.linewidth": 1.6,
})

def save(fig, name):
    """Save a figure as vector PDF (for LaTeX) and PNG (for the HTML export)
    fig.savefig(FIG / f"{name}.pdf")
    fig.savefig(FIG / f"{name}.png", dpi=200)
    print(f" saved figures/{name}.pdf")

RESULTS = {} # every number quoted in the paper is registered here
def reg(key, value):
    RESULTS[key] = value
    return value
```

3. Dataset

Source. *Diabetes 130-US hospitals for years 1999–2008*, UCI Machine Learning Repository (ID 296), donated by Strack et al. (2014), CC BY 4.0. 101,766 inpatient encounters, 130 hospitals, 10 years, 50 variables.

Why this dataset. (i) It is the only large, public, de-identified inpatient dataset that simultaneously carries demographics, diagnoses (ICD-9), laboratory results (HbA1c, glucose), 23 medication variables, administrative attributes and an explicit 30-day readmission label; (ii) it carries a patient identifier, which makes the patient-level validation question answerable; (iii) it is a widely used benchmark, so our results are directly comparable with published work; (iv) it exhibits exactly the pathologies named in the problem statement — missingness, class imbalance and many interacting variables.

```
In [2]: t0 = time.time()
df_raw = pd.read_csv(DATA, low_memory=False)
df_raw = df_raw.replace("?", np.nan)
print("Raw shape:", df_raw.shape)
reg("n_raw_encounters", int(df_raw.shape[0]))
reg("n_raw_features", int(df_raw.shape[1]) - 1)
reg("n_raw_patients", int(df_raw.patient_nbr.nunique()))
df_raw.head(3)
```

Raw shape: (101766, 50)

```
Out[2]:
```

	encounter_id	patient_nbr	race	gender	age	weight	admission
0	2278392	8222157	Caucasian	Female	[0-10)	NaN	
1	149190	55629189	Caucasian	Female	[10-20)	NaN	
2	64410	86047875	AfricanAmerican	Female	[20-30)	NaN	

3 rows × 50 columns

```
In [3]: # Structure of the raw table
overview = pd.DataFrame({
    "dtype": df_raw.dtypes.astype(str),
    "n_unique": df_raw.nunique(),
    "missing_%": (df_raw.isna().mean() * 100).round(2),
})
print(overview.to_string())
```

	dtype	n_unique	missing_%
encounter_id	int64	101766	0.00
patient_nbr	int64	71518	0.00
race	str	5	2.23
gender	str	3	0.00
age	str	10	0.00
weight	str	9	96.86
admission_type_id	int64	8	0.00
discharge_disposition_id	int64	26	0.00
admission_source_id	int64	17	0.00
time_in_hospital	int64	14	0.00
payer_code	str	17	39.56
medical_specialty	str	72	49.08
num_lab_procedures	int64	118	0.00
num_procedures	int64	7	0.00
num_medications	int64	75	0.00
number_outpatient	int64	39	0.00
number_emergency	int64	33	0.00
number_inpatient	int64	21	0.00
diag_1	str	716	0.02
diag_2	str	748	0.35
diag_3	str	789	1.40
number_diagnoses	int64	16	0.00
max_glu_serum	str	3	94.75
A1Cresult	str	3	83.28
metformin	str	4	0.00
repaglinide	str	4	0.00
nateglinide	str	4	0.00
chlorpropamide	str	4	0.00
glimepiride	str	4	0.00
acetohexamide	str	2	0.00
glipizide	str	4	0.00
glyburide	str	4	0.00
tolbutamide	str	2	0.00
pioglitazone	str	4	0.00
rosiglitazone	str	4	0.00
acarbose	str	4	0.00
miglitol	str	4	0.00
troglitazone	str	2	0.00
tolazamide	str	3	0.00
examide	str	1	0.00
citoglipton	str	1	0.00
insulin	str	4	0.00
glyburide-metformin	str	4	0.00
glipizide-metformin	str	2	0.00
glimepiride-pioglitazone	str	2	0.00
metformin-rosiglitazone	str	2	0.00
metformin-pioglitazone	str	2	0.00
change	str	2	0.00
diabetesMed	str	2	0.00
readmitted	str	3	0.00

3.1 Cohort construction

Following the original data statement we remove encounters that could not, by construction, be followed by a readmission: discharge to hospice or death (`discharge_disposition_id` in {11, 13, 14, 19, 20, 21}). We also drop the three encounters with `gender = "Unknown/Invalid"`. Unlike much of the literature we keep **all** remaining encounters of a patient rather than only the first — discarding repeat admissions throws away the prior-utilisation signal that is the strongest predictor — and instead control the resulting dependence at the *splitting* stage.

```
In [4]: EXPIRED_HOSPICE = {11, 13, 14, 19, 20, 21}
df = df_raw.copy()
n0 = len(df)
df = df[~df.discharge_disposition_id.isin(EXPIRED_HOSPICE)]
n1 = len(df)
df = df[df.gender != "Unknown/Invalid"]
n2 = len(df)
print(f"encounters: {n0} -> {n1} (expired/hospice removed: {n0-n1}) -> {n2}")
reg("n_excluded_expired", int(n0 - n1))
reg("n_excluded_gender", int(n1 - n2))
reg("n_cohort", int(n2))
reg("n_cohort_patients", int(df.patient_nbr.nunique()))

# Binary target: readmission within 30 days
df["y"] = (df["readmitted"] == "<30").astype(int)
prev = df.y.mean()
print(f"cohort: {len(df):,} encounters from {df.patient_nbr.nunique():,} patients")
print(f"30-day readmission prevalence: {prev:.4f} ({df.y.sum():,} positives)")
print(f"imbalance ratio 1: {(1-prev)/prev:.1f}")
reg("prevalence", float(prev))
reg("n_positives", int(df.y.sum()))
reg("imbalance_ratio", float((1 - prev) / prev))
reg("encounters_per_patient", float(len(df) / df.patient_nbr.nunique()))
reg("pct_repeat_encounters", float((df.patient_nbr.duplicated(keep=False)).n
```

```
encounters: 101766 -> 99343 (expired/hospice removed: 2423) -> 99340 (invalid gender: 3)
cohort: 99,340 encounters from 69,987 patients
30-day readmission prevalence: 0.1139 (11,314 positives)
imbalance ratio 1:7.8
```

```
Out[4]: 45.997584054761425
```

3.2 Missing-data analysis

Missingness is far from negligible. The key methodological question is whether it is ignorable. We test each incomplete variable's *missingness indicator* against the outcome with a χ^2 test: a significant association means the missingness is itself informative and must be modelled as a category rather than imputed away.

```
In [5]: miss = (df.isna().mean() * 100).sort_values(ascending=False)
miss = miss[miss > 0]
rows = []
for col, pct in miss.items():
    ind = df[col].isna().astype(int)
    ct = pd.crosstab(ind, df.y)
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    rate_missing = df.loc[ind == 1, "y"].mean() * 100
    rate_present = df.loc[ind == 0, "y"].mean() * 100
    rows.append([col, pct, rate_missing, rate_present, chi2, p])
missing_tbl = pd.DataFrame(rows, columns=["Variable", "Missing (%)", "Readm.
                                         "Readm. rate | observed (%)", "chi
missing_tbl = missing_tbl.sort_values("Missing (%)", ascending=False).reset_
print(missing_tbl.to_string(index=False))
reg("weight_missing_pct", float(missing_tbl.set_index("Variable").loc["weigh
reg("payer_missing_pct", float(missing_tbl.set_index("Variable").loc["payer_
reg("spec_missing_pct", float(missing_tbl.set_index("Variable").loc["medical
reg("race_missing_pct", float(missing_tbl.set_index("Variable").loc["race",
```

	Variable	Missing (%)	Readm. rate missing (%)	Readm. rate obs
	erved (%)	chi2	p	
	weight	96.854238		11.389077
11.392000	0.000000	1.000000e+00		
	max_glu_serum	94.813771		11.314605
12.752329	9.863106	1.686262e-03		
	A1Cresult	83.054157		11.685211
9.938220	42.108627	8.634145e-11		
	medical_specialty	48.936984		11.834040
10.962820	18.583775	1.625986e-05		
	payer_code	39.658748		11.691246
11.190631	5.853942	1.554220e-02		
	race	2.246829		8.422939
11.457346	19.606920	9.512405e-06		
	diag_3	1.428428		6.412967
11.461280	34.823750	3.609435e-09		
	diag_2	0.358365		8.146067
11.400832	3.407975	6.488203e-02		
	diag_1	0.020133		25.000000
11.386428	2.446987	1.177507e-01		

Out[5]: 2.246829071874371

```
In [6]: # LaTeX table: missingness + informativeness
def fmt_p(p):
    return "$<$0.001" if p < 1e-3 else f"{p:.3f}"

mt = missing_tbl.copy()
mt["Variable"] = mt["Variable"].str.replace("_", r"\_", regex=False)
lines = [r"\begin{tabular}{lrrrr}", r"\hline\noalign{\smallskip}",
         r"Variable & Missing (\%) & Readm. if missing (\%) & Readm. if obse",
         r"\noalign{\smallskip}\hline\noalign{\smallskip}"]
for _, r in mt.iterrows():
    lines.append(f"{r['Variable']} & {r['Missing (%)']:.1f} & {r['Readm. rat",
                 f"{r['Readm. rate | observed (%)']:.1f} & {fmt_p(r['p'])} \")
lines += [r"\noalign{\smallskip}\hline", r"\end{tabular}"]
```



```

        return "Other"
    if 250 <= v < 251:
        return "Diabetes"
    if (390 <= v < 460) or int(v) == 785:
        return "Circulatory"
    if (460 <= v < 520) or int(v) == 786:
        return "Respiratory"
    if (520 <= v < 580) or int(v) == 787:
        return "Digestive"
    if 800 <= v < 1000:
        return "Injury"
    if 710 <= v < 740:
        return "Musculoskeletal"
    if (580 <= v < 630) or int(v) == 788:
        return "Genitourinary"
    if 140 <= v < 240:
        return "Neoplasms"
    return "Other"

ADM_TYPE = {1: "Emergency", 2: "Urgent", 3: "Elective", 4: "Newborn", 7: "Tr
ADM_SRC = {1: "Physician referral", 2: "Clinic referral", 3: "HMO referral",
def adm_src_group(v):
    if v in ADM_SRC:
        return "Emergency room" if v == 7 else "Referral"
    if v in {4, 5, 6, 10, 18, 22, 25, 26}:
        return "Transfer"
    if v in {11, 12, 13, 14, 23, 24}:
        return "Delivery/newborn"
    return "Unknown"

def disch_group(v):
    if v == 1:
        return "Home"
    if v in {6, 8}:
        return "Home with home health"
    if v in {2, 3, 4, 5, 22, 23, 24, 15, 16, 17, 30, 27, 28, 29, 9, 10, 12}:
        return "Transfer to another facility"
    if v == 7:
        return "Against medical advice"
    return "Unknown/Other"

MEDS = ['metformin', 'repaglinide', 'nateglinide', 'chlorpropamide', 'glimepiride',
        'glipizide', 'glyburide', 'tolbutamide', 'pioglitazone', 'rosiglitazone',
        'troglitazone', 'tolazamide', 'examide', 'citoglipton', 'insulin', 'glybu
        'glipizide-metformin', 'glimepiride-pioglitazone', 'metformin-rosiglit

f = pd.DataFrame(index=df.index)

# --- demographic ---
age_mid = {f"[{i}-{i+10})": i + 5 for i in range(0, 100, 10)}
f["age_mid"] = df.age.map(age_mid)
f["race"] = df.race.fillna("Missing")
f["gender"] = df.gender

# --- administrative / non-clinical ---
f["admission_type"] = df.admission_type_id.map(ADM_TYPE).fillna("Unknown")

```

```

f["admission_source"] = df.admission_source_id.map(adm_src_group)
f["discharge_group"] = df.discharge_disposition_id.map(disch_group)
f["payer_code"] = df.payer_code.fillna("Missing")
top_spec = df.medical_specialty.value_counts().head(15).index
f["medical_specialty"] = np.where(df.medical_specialty.isna(), "Missing",
                                  np.where(df.medical_specialty.isin(top_spec),
                                             df.medical_specialty, "Missing"))

# --- clinical ---
f["time_in_hospital"] = df.time_in_hospital
f["num_lab_procedures"] = df.num_lab_procedures
f["num_procedures"] = df.num_procedures
f["num_medications"] = df.num_medications
f["number_diagnoses"] = df.number_diagnoses
f["diag_1_group"] = df.diag_1.map(icd9_group)
f["diag_2_group"] = df.diag_2.map(icd9_group)
f["diag_3_group"] = df.diag_3.map(icd9_group)
f["A1C"] = df.A1Cresult.fillna("Not measured")
f["glucose"] = df.max_glu_serum.fillna("Not measured")
f["A1C_tested"] = df.A1Cresult.notna().astype(int)
f["glucose_tested"] = df.max_glu_serum.notna().astype(int)
f["lab_per_day"] = (df.num_lab_procedures / df.time_in_hospital).round(3)
f["meds_per_day"] = (df.num_medications / df.time_in_hospital).round(3)

# --- medication ---
med = df[MEDS]
f["n_meds_active"] = (med != "No").sum(axis=1)
f["n_med_changes"] = med.isin(["Up", "Down"]).sum(axis=1)
f["insulin"] = df.insulin
f["change"] = df.change
f["diabetesMed"] = df.diabetesMed

# --- prior utilisation ---
f["number_outpatient"] = df.number_outpatient
f["number_emergency"] = df.number_emergency
f["number_inpatient"] = df.number_inpatient
f["service_utilization"] = df.number_outpatient + df.number_emergency + df.number_inpatient
f["any_prior_inpatient"] = (df.number_inpatient > 0).astype(int)
f["any_prior_emergency"] = (df.number_emergency > 0).astype(int)
# index of this encounter within the patient's own (temporally ordered) history
f["prior_encounters"] = df.sort_values("encounter_id").groupby("patient_nbr").count().shift(1)

y = df.y.values
groups = df.patient_nbr.values
print("engineered feature matrix:", f.shape)
NUMERIC = [c for c in f.columns if pd.api.types.is_numeric_dtype(f[c])]
CATEG = [c for c in f.columns if not pd.api.types.is_numeric_dtype(f[c])]
print(f"{len(NUMERIC)} numeric, {len(CATEG)} categorical -> ", len(NUMERIC), len(CATEG))
reg("n_engineered", int(f.shape[1]))
reg("n_numeric", len(NUMERIC)); reg("n_categorical", len(CATEG))

FEATURE_BLOCKS = {
    "Demographic": ["age_mid", "race", "gender"],
    "Administrative": ["admission_type", "admission_source", "discharge_group", "payer_code", "medical_specialty"],
    "Clinical": ["time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications", "number_diagnoses", "diag_1_group", "diag_2_group", "diag_3_group", "A1C_tested", "glucose_tested", "lab_per_day", "meds_per_day"]
}

```

```

    "Medication": ["n_meds_active", "n_med_changes", "insulin", "change", "c
    "Prior utilisation": ["number_outpatient", "number_emergency", "number_i
                        "service_utilization", "any_prior_inpatient", "any
                        "prior_encounters"],
}
assert sorted(sum(FEATURE_BLOCKS.values(), [])) == sorted(f.columns.tolist())

```

engineered feature matrix: (99340, 34)
19 numeric, 15 categorical -> 34 variables

4. Exploratory data analysis

4.1 Outcome, imbalance and patient clustering

```

In [8]: fig, axes = plt.subplots(1, 3, figsize=(7.2, 2.5))

ax = axes[0]
counts = df.readmitted.value_counts().reindex(["NO", ">30", "<30"])
bars = ax.bar(["No readm.", "Readm. >30d", "Readm. <30d"], counts.values / 1
              color=[C["muted"], C["blue"], C["orange"]], width=0.62, zorder
for b, v in zip(bars, counts.values):
    ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 1.2, f"{v/1000:.
            ha="center", fontsize=8, color=C["ink"])
ax.set_ylabel("Encounters (thousands)"); ax.set_title("(a) Outcome distribut
ax.set_ylim(0, counts.max() / 1000 * 1.18); ax.tick_params(axis="x", labelrc

ax = axes[1]
enc_per_pat = df.groupby("patient_nbr").size().value_counts().sort_index()
k = enc_per_pat.index.values; v = enc_per_pat.values
keep = k <= 6
kk = list(k[keep].astype(str)) + ["7+"]
vv = list(v[keep]) + [v[~keep].sum()]
ax.bar(kk, np.array(vv) / 1000, color=C["blue"], width=0.66, zorder=3)
ax.set_xlabel("Encounters contributed by a patient"); ax.set_ylabel("Patient
ax.set_title("(b) Patient clustering", loc="left")
ax.text(0.97, 0.9, f"{RESULTS['pct_repeat_encounters']:.0f}% of encounters\r
        transform=ax.transAxes, ha="right", fontsize=7.5, color=C["ink2"])

ax = axes[2]
# readmission rate as a function of the patient's encounter index
tmp = pd.DataFrame({"prior": f.prior_encounters.clip(upper=5), "y": y})
g = tmp.groupby("prior").y.agg(["mean", "count", "sum"])
se = np.sqrt(g["mean"] * (1 - g["mean"]) / g["count"])
ax.errorbar(g.index, g["mean"] * 100, yerr=1.96 * se * 100, marker="o", ms=4
            color=C["orange"], capsize=2.5, lw=1.6, zorder=3)
ax.axhline(prev * 100, color=C["ink2"], ls=(0, (4, 3)), lw=1.0, zorder=2)
ax.text(0.03, prev * 100 + 0.6, "cohort mean", fontsize=7, color=C["ink2"],
ax.set_xlabel("Prior encounters of same patient"); ax.set_ylabel("30-day rea
ax.set_xticks(range(6)); ax.set_xticklabels(["0", "1", "2", "3", "4", "5+"])
ax.set_title("(c) Risk vs. patient history", loc="left")
fig.tight_layout()
save(fig, "fig_outcome"); plt.close(fig)

```

```
reg("rate_prior0", float(g.loc[0, "mean"] * 100))
reg("rate_prior5", float(g.loc[5, "mean"] * 100))
```

saved figures/fig_outcome.pdf

Out[8]: 30.653674380836378

Panel (c) is the core motivation for patient-level validation: risk rises monotonically with the number of previous encounters the same patient contributes to the dataset. A random split therefore lets the model memorise *patients*, not learn *risk*.

```
In [9]: # ---- Univariate association screening -----
def cramers_v(ct):
    chi2 = stats.chi2_contingency(ct)[0]
    n = ct.values.sum()
    r, k = ct.shape
    phi2 = chi2 / n
    return np.sqrt(phi2 / min(r - 1, k - 1))

rows = []
for c in CATEG:
    ct = pd.crosstab(f[c], y)
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    rows.append([c, "categorical", cramers_v(ct), p, ""])
for c in NUMERIC:
    a, b = f.loc[y == 1, c], f.loc[y == 0, c]
    u, p = stats.mannwhitneyu(a, b, alternative="two-sided")
    # rank-biserial correlation as effect size
    rbc = 1 - 2 * u / (len(a) * len(b))
    rows.append([c, "numeric", abs(rbc), p, f"{a.median():.1f} vs {b.median():.1f}"])
assoc = pd.DataFrame(rows, columns=["feature", "type", "effect", "p", "media"])
assoc["p_bonf"] = np.minimum(assoc.p * len(assoc), 1.0)
assoc = assoc.sort_values("effect", ascending=False).reset_index(drop=True)
print(assoc.head(15).to_string(index=False))
reg("n_signif_bonf", int((assoc.p_bonf < 0.05).sum()))
reg("n_tests", int(len(assoc)))
```

	feature	type	effect	p	medians	p
_bonf						
0e+00	number_inpatient	numeric	0.214528	0.000000e+00	0.0 vs 0.0	0.000000
0e+00	service_utilization	numeric	0.208397	0.000000e+00	1.0 vs 0.0	0.000000
0e+00	prior_encounters	numeric	0.186684	0.000000e+00	0.0 vs 0.0	0.000000
0e+00	any_prior_inpatient	numeric	0.184971	0.000000e+00	0.0 vs 0.0	0.000000
7e-61	time_in_hospital	numeric	0.095394	1.587928e-62	4.0 vs 4.0	5.39895
e-173	discharge_group	categorical	0.090525	6.863832e-175		2.333703
0e-58	number_diagnoses	numeric	0.088026	2.632621e-59	9.0 vs 8.0	8.95091
8e-50	num_medications	numeric	0.087191	9.394582e-52	16.0 vs 15.0	3.19415
5e-90	number_emergency	numeric	0.063966	1.244690e-91	0.0 vs 0.0	4.23194
5e-84	any_prior_emergency	numeric	0.061791	6.622366e-86	0.0 vs 0.0	2.25160
4e-30	n_med_changes	numeric	0.052667	3.040570e-32	0.0 vs 0.0	1.03379
9e-18	lab_per_day	numeric	0.051835	2.480650e-19	10.2 vs 11.0	8.43420
0e-41	insulin	categorical	0.044782	6.221589e-43		2.11534
6e-13	num_lab_procedures	numeric	0.043830	2.910622e-14	45.0 vs 44.0	9.89611
5e-13	age_mid	numeric	0.043246	2.057331e-14	65.0 vs 65.0	6.99492

Out[9]: 34

```
In [10]: fig, ax = plt.subplots(figsize=(4.6, 3.4))
top = assoc.head(14).iloc[::-1]
cols = [C["orange"] if t == "numeric" else C["blue"] for t in top.type]
ax.barh(top.feature.str.replace("_", " "), top.effect, color=cols, height=0.1)
ax.set_xlabel("Effect size (Cramér's  $V$  / |rank-biserial  $r$ |)")
ax.set_title("Strongest univariate associations with 30-day readmission", loc="left")
h = [plt.Rectangle((0, 0), 1, 1, color=C["blue"]), plt.Rectangle((0, 0), 1, 1, color=C["orange"])]
ax.legend(h, ["Categorical (Cramér's  $V$ )", "Numeric (rank-biserial)"], loc="left")
fig.tight_layout(); save(fig, "fig_assoc"); plt.close(fig)
```

saved figures/fig_assoc.pdf

4.2 Risk stratification by candidate predictors

```
In [11]: def rate_by(series, ax, title, order=None, rot=0, topn=None):
    t = pd.DataFrame({"g": series.values, "y": y})
    agg = t.groupby("g").y.agg(["mean", "count"])
    agg = agg[agg["count"] >= 100]
    if topn:
        agg = agg.sort_values("count", ascending=False).head(topn)
    agg = agg.reindex(order) if order is not None else agg.sort_values("mean")
```

```

agg = agg.dropna()
se = np.sqrt(agg["mean"] * (1 - agg["mean"]) / agg["count"])
ax.barh(range(len(agg)), agg["mean"] * 100, xerr=1.96 * se * 100,
        color=C["blue"], height=0.62, zorder=3,
        error_kw=dict(ecolor=C["ink2"], lw=0.8, capsize=2))
ax.set_yticks(range(len(agg))); ax.set_yticklabels([str(i) for i in agg.
ax.axvline(prev * 100, color=C["orange"], ls=(0, (4, 2)), lw=1.2, zorder
ax.set_title(title, loc="left", fontsize=9); ax.set_xlabel("30-day readm
return agg

fig, axes = plt.subplots(2, 2, figsize=(7.2, 5.4))
a1 = rate_by(f.discharge_group, axes[0, 0], "(a) Discharge disposition")
a2 = rate_by(pd.cut(f.number_inpatient, [-1, 0, 1, 2, 3, 100],
        labels=["0", "1", "2", "3", "4+"], axes[0, 1], "(b) Pri
        order=["0", "1", "2", "3", "4+"])
a3 = rate_by(f.A1C, axes[1, 0], "(c) HbA1c result", order=["Not measured", "
a4 = rate_by(f.diag_1_group, axes[1, 1], "(d) Primary diagnosis chapter", to
fig.tight_layout()
fig.text(0.5, -0.012, "Dashed vertical line = cohort mean (11.4%); bars show
        ha="center", fontsize=7.5, color=C["ink2"])
save(fig, "fig_riskstrat"); plt.close(fig)

reg("rate_inp0", float(a2.loc["0", "mean"] * 100)); reg("rate_inp4", float(a
reg("rate_disch_max", float(a1["mean"].max() * 100)); reg("rate_disch_max_lb
reg("rate_alc_notmeas", float(a3.loc["Not measured", "mean"] * 100))
reg("rate_alc_norm", float(a3.loc["Norm", "mean"] * 100))

```

saved figures/fig_riskstrat.pdf

Out[11]: 9.772450223486388

```

In [12]: # ---- Correlation structure among numeric predictors (interacting variables
corr = f[NUMERIC].corr(method="spearman")
fig, ax = plt.subplots(figsize=(5.6, 4.9))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
M = np.ma.masked_array(corr.values, mask)
cmap = mpl.colors.LinearSegmentedColormap.from_list("div", ["#2a78d6", "#f2f
im = ax.imshow(M, cmap=cmap, vmin=-1, vmax=1)
ax.set_xticks(range(len(NUMERIC))); ax.set_yticks(range(len(NUMERIC)))
lbl = [c.replace("_", " ") for c in NUMERIC]
ax.set_xticklabels(lbl, rotation=90, fontsize=6.5); ax.set_yticklabels(lbl,
ax.grid(False)
cb = fig.colorbar(im, ax=ax, fraction=0.045, pad=0.03); cb.set_label("Spearman
cb.ax.tick_params(labelsize=7)
ax.set_title("Correlation among numeric predictors", loc="left", fontsize=9)
fig.tight_layout(); save(fig, "fig_corr"); plt.close(fig)

cc = corr.where(~np.eye(len(corr), dtype=bool)).abs().stack()
reg("max_corr_pair", [str(cc.idxmax()[0]), str(cc.idxmax()[1]), float(cc.max
print("strongest |rho| pair:", RESULTS["max_corr_pair"])

```

saved figures/fig_corr.pdf

strongest |rho| pair: ['number_emergency', 'any_prior_emergency', 0.99839350
52069791]

5. Experimental design

5.1 Splitting strategy

The unit of analysis is the *encounter*, but encounters are nested within *patients*. We therefore use `GroupShuffleSplit` on `patient_nbr` for the train/test split and `StratifiedGroupKFold` for cross-validation, guaranteeing that **no patient appears in both folds**. As a deliberate counterfactual (objective **O2**) we repeat the entire protocol with a naive stratified random split of encounters and report the difference as *optimism*.

```
In [13]: from sklearn.model_selection import (GroupShuffleSplit, StratifiedGroupKFold,
                                             RandomizedSearchCV, train_test_split)

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (roc_auc_score, average_precision_score, brier_score_function,
                             roc_curve, precision_recall_curve, f1_score, recall_score,
                             precision_score, confusion_matrix, balanced_accuracy_score)
from sklearn.calibration import calibration_curve
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

gss = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=RANDOM_STATE)
tr_idx, te_idx = next(gss.split(f, y, groups))
X_tr, X_te = f.iloc[tr_idx], f.iloc[te_idx]
y_tr, y_te = y[tr_idx], y[te_idx]
g_tr = groups[tr_idx]
print(f"train {X_tr.shape[0]:,} enc / {pd.Series(g_tr).unique().shape[0]:,} patients")
print(f"test {X_te.shape[0]:,} enc / {pd.Series(groups[te_idx]).unique().shape[0]:,} patients")
print("patient overlap:", len(set(groups[tr_idx]) & set(groups[te_idx])))
reg("n_train", int(len(tr_idx))); reg("n_test", int(len(te_idx)))
reg("n_train_pat", int(pd.Series(g_tr).unique().shape[0])); reg("n_test_pat", int(pd.Series(groups[te_idx]).unique().shape[0]))
reg("prev_train", float(y_tr.mean())); reg("prev_test", float(y_te.mean()))

train 74,583 enc / 52,490 patients (prev 0.1143)
test 24,757 enc / 17,497 patients (prev 0.1127)
patient overlap: 0
```

Out[13]: 0.11265500666478168

```
In [14]: def make_prep(scale=True, cols=None):
    """One-hot for categoricals (rare levels folded), standardisation for numericals
    cols = cols if cols is not None else list(f.columns)
    num = [c for c in cols if c in NUMERIC]
    cat = [c for c in cols if c in CATEG]
    return ColumnTransformer([
        ("num", StandardScaler() if scale else "passthrough", num),
        ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_freq=1,
                              sparse_output=False), cat),
    ], remainder="drop")
```

```
SPW = float((y_tr == 0).sum() / (y_tr == 1).sum()) # scale_pos_weight for
print("scale_pos_weight =", round(SPW, 2))
```

scale_pos_weight = 7.75

5.2 Metrics

With 11% prevalence, accuracy is uninformative and AUROC is insensitive to the operating region that matters. We report:

- **AUROC** — overall ranking;
- **AUPRC** — ranking in the positive class (no-skill value = prevalence = 0.114);
- **Brier score** and **ECE** — calibration, i.e. whether a "20% risk" really means 20%;
- **Sensitivity / PPV at a 10% alert rate** — the realistic deployment constraint where a transitional-care team can follow up the top decile of risk;
- **Net benefit** (decision-curve analysis) — clinical utility at a chosen risk threshold.

```
In [15]: def ece(y_true, p, bins=10):
    """Expected calibration error (equal-width bins)."""
    edges = np.linspace(0, 1, bins + 1)
    idx = np.clip(np.digitize(p, edges[1:-1]), 0, bins - 1)
    e = 0.0
    for b in range(bins):
        m = idx == b
        if m.sum():
            e += m.mean() * abs(y_true[m].mean() - p[m].mean())
    return e

def metrics_at_alert(y_true, p, rate=0.10):
    """Flag exactly the top `rate` fraction of encounters by predicted risk.

    Selection is by rank rather than by thresholding: isotonic recalibration
    collapses predictions onto ~100 distinct values, and `p >= threshold` wc
    then flag far more encounters than the intended budget (4,115 instead of
    2,476 here) and overstate sensitivity.
    """
    k = int(np.ceil(len(p) * rate))
    order = np.argsort(p, kind="stable")[:, -1][:k]
    yhat = np.zeros(len(p), dtype=int)
    yhat[order] = 1
    thr = float(p[order[-1]])
    tn, fp, fn, tp = confusion_matrix(y_true, yhat).ravel()
    return dict(sens=tp / (tp + fn), ppv=tp / (tp + fp) if (tp + fp) else np.nan,
                nne=(tp + fp) / tp if tp else np.nan, threshold=thr)

def net_benefit(y_true, p, pt):
    yhat = (p >= pt).astype(int)
    tp = ((yhat == 1) & (y_true == 1)).sum(); fp = ((yhat == 1) & (y_true == 0)).sum()
    n = len(y_true)
    return tp / n - (fp / n) * (pt / (1 - pt))
```

```

def evaluate(y_true, p, name=""):
    m = dict(model=name,
              AUROC=roc_auc_score(y_true, p),
              AUPRC=average_precision_score(y_true, p),
              Brier=brier_score_loss(y_true, p),
              ECE=ece(y_true, p))
    m.update({f"{k}@10%": v for k, v in metrics_at_alert(y_true, p).items()})
    return m

def boot_ci(y_true, p, fn, n_boot=400, seed=0):
    r = np.random.default_rng(seed)
    n = len(y_true)
    vals = []
    for _ in range(n_boot):
        i = r.integers(0, n, n)
        if y_true[i].sum() == 0:
            continue
        vals.append(fn(y_true[i], p[i]))
    return float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))

```

6. Model development

Four model families are tuned under an **identical** protocol: 3-fold

`StratifiedGroupKFold` randomised search on the training patients only, optimising average precision (AUPRC). Class imbalance is handled *inside* the learner (`class_weight / scale_pos_weight`) rather than by resampling — Section 8 tests that choice explicitly.

In [16]: `cv = StratifiedGroupKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)`

```

def tuned(name, est, grid, n_iter, prep_scale=True):
    pipe = Pipeline([("prep", make_prep(scale=prep_scale)), ("clf", est)])
    search = RandomizedSearchCV(pipe, {f"clf__{k}": v for k, v in grid.items()},
                                n_iter=n_iter, scoring="average_precision",
                                random_state=RANDOM_STATE, n_jobs=1, refit=True)

    t = time.time()
    search.fit(X_tr, y_tr, groups=g_tr)
    print(f"{name:20s} CV AUPRC={search.best_score_:.4f} ({time.time()-t:.6s})")
    print(f"    best: { {k.replace('clf__', ''): v for k, v in search.best_params_.items() } }")
    return search

```

SEARCHES = {}

```

SEARCHES["Logistic regression"] = tuned(
    "Logistic regression",
    LogisticRegression(max_iter=2000, class_weight="balanced", solver="liblinear"),
    {"C": [0.003, 0.01, 0.03, 0.1, 0.3, 1.0], "penalty": ["l2", "l1"]}, n_iter=100)

```

Logistic regression CV AUPRC=0.2046 (45s)
best: {'penalty': 'l2', 'C': 0.01}

In [17]: `SEARCHES["Random forest"] = tuned(
 "Random forest",`

```
RandomForestClassifier(n_estimators=250, class_weight="balanced_subsample",
                      random_state=RANDOM_STATE, n_jobs=4),
{"max_depth": [8, 12, 18, None], "min_samples_leaf": [5, 20, 50],
 "max_features": ["sqrt", 0.3]}, n_iter=6, prep_scale=False)
```

Random forest CV AUPRC=0.2158 (147s)
 best: {'min_samples_leaf': 50, 'max_features': 0.3, 'max_depth': 12}

```
In [18]: SEARCHES["XGBoost"] = tuned(
    "XGBoost",
    XGBClassifier(n_estimators=500, scale_pos_weight=SPW, tree_method="hist",
                 eval_metric="aucpr", random_state=RANDOM_STATE, n_jobs=4,
                 early_stopping_rounds=None),
    {"max_depth": [3, 4, 6, 8], "learning_rate": [0.02, 0.05, 0.1],
     "subsample": [0.7, 0.9], "colsample_bytree": [0.6, 0.8, 1.0],
     "min_child_weight": [1, 5, 20], "reg_lambda": [1.0, 5.0, 20.0]}, n_iter
```

XGBoost CV AUPRC=0.2132 (82s)
 best: {'subsample': 0.7, 'reg_lambda': 20.0, 'min_child_weight': 20, 'max_depth': 3, 'learning_rate': 0.02, 'colsample_bytree': 1.0}

```
In [19]: SEARCHES["LightGBM"] = tuned(
    "LightGBM",
    LGBMClassifier(n_estimators=500, class_weight="balanced", random_state=RANDOM_STATE,
                  n_jobs=4, verbose=-1),
    {"num_leaves": [15, 31, 63], "learning_rate": [0.02, 0.05, 0.1],
     "min_child_samples": [20, 50, 200], "subsample": [0.8, 1.0],
     "colsample_bytree": [0.6, 0.8, 1.0], "reg_lambda": [0.0, 1.0, 10.0]}, r
```

LightGBM CV AUPRC=0.2130 (55s)
 best: {'subsample': 0.8, 'reg_lambda': 10.0, 'num_leaves': 15, 'min_child_samples': 200, 'learning_rate': 0.02, 'colsample_bytree': 1.0}

```
In [20]: # Baseline: predicts the training prevalence for everyone (no-skill reference)
base = DummyClassifier(strategy="prior").fit(X_tr, y_tr)

MODELS = {"Baseline (prevalence)": base}
MODELS.update({k: v.best_estimator_ for k, v in SEARCHES.items()})
PRED = {k: m.predict_proba(X_te)[:, 1] for k, m in MODELS.items()}

cvres = {k: float(v.best_score_) for k, v in SEARCHES.items()}
reg("cv_auprc", cvres)
reg("best_params", {k: {kk.replace("clf_", ""): (str(vv)) for kk, vv in v.best_params_.items()}
                    for k, v in SEARCHES.items()})
print(json.dumps(cvres, indent=1))
```

```
{
  "Logistic regression": 0.20457184266560516,
  "Random forest": 0.21582619459506236,
  "XGBoost": 0.21316580423796902,
  "LightGBM": 0.2130113473258628
}
```

```
In [21]: # Persist fitted pipelines and the exact split so every result below is reproducible
import joblib
(ROOT / "artifacts").mkdir(exist_ok=True)
joblib.dump({"models": MODELS, "pred": PRED, "tr_idx": tr_idx, "te_idx": te_idx,
            "searches_best": {k: v.best_params_ for k, v in SEARCHES.items()}}
```

```

        "cv_auprc": cvres, "f": f, "y": y, "groups": groups, "results":
        ROOT / "artifacts" / "models.joblib")
print("cached ->", ROOT / "artifacts" / "models.joblib")

```

cached -> /home/user/RM_Lab/artifacts/models.joblib

7. Held-out performance

All models are evaluated once, on the patient-disjoint test set.

```

In [22]: rows = []
        for name, p in PRED.items():
            m = evaluate(y_te, p, name)
            lo, hi = boot_ci(y_te, p, roc_auc_score)
            m["AUROC_lo"], m["AUROC_hi"] = lo, hi
            lo, hi = boot_ci(y_te, p, average_precision_score)
            m["AUPRC_lo"], m["AUPRC_hi"] = lo, hi
            rows.append(m)
        res = pd.DataFrame(rows).set_index("model")
        print(res.round(4).to_string())
        reg("test_results", json.loads(res.reset_index().to_json(orient="records")))

        BEST = res.drop(index="Baseline (prevalence)").AUPRC.idxmax()
        print("\nbest model by test AUPRC:", BEST)
        reg("best_model", BEST)

```

		AUROC	AUPRC	Brier	ECE	sens@10%	ppv@10%	nn
e@10%	threshold@10%	AUROC_lo	AUROC_hi	AUPRC_lo	AUPRC_hi			
model								
Baseline (prevalence)		0.5000	0.1127	0.1000	0.0016	0.0875	0.0985	1
0.1475	0.1143	0.5000	0.5000	0.1091	0.1166			
Logistic regression		0.6645	0.2057	0.2273	0.3554	0.2119	0.2387	
4.1895	0.6506	0.6538	0.6753	0.1935	0.2203			
Random forest		0.6742	0.2187	0.2027	0.3195	0.2273	0.2561	
3.9054	0.6116	0.6647	0.6854	0.2060	0.2349			
XGBoost		0.6748	0.2166	0.2231	0.3498	0.2273	0.2561	
3.9054	0.6433	0.6657	0.6856	0.2038	0.2320			
LightGBM		0.6770	0.2228	0.2197	0.3434	0.2349	0.2645	
3.7802	0.6458	0.6677	0.6881	0.2095	0.2382			

best model by test AUPRC: LightGBM

Out[22]: 'LightGBM'

```

In [23]: # Paired bootstrap: does the best model beat regularised logistic regression
        def paired_boot(pa, pb, metric=average_precision_score, n_boot=800, seed=1):
            r = np.random.default_rng(seed)
            d = []
            for _ in range(n_boot):
                i = r.integers(0, len(y_te), len(y_te))
                if y_te[i].sum() == 0:
                    continue
                d.append(metric(y_te[i], pa[i]) - metric(y_te[i], pb[i]))
            d = np.array(d)
            return float(d.mean()), float(np.percentile(d, 2.5)), float(np.percentile(d, 97.5))

```

```

for metric, mname in [(average_precision_score, "AUPRC"), (roc_auc_score, "AUC")]:
    diff, lo, hi, pval = paired_boot(PRED[BEST], PRED["Logistic regression"])
    print(f"{BEST} - LR {mname}: {diff:+.4f} 95% CI [{lo:+.4f}, {hi:+.4f}]")
    reg(f"delta_{mname}_best_vs_lr", dict(diff=diff, lo=lo, hi=hi, p=pval))

```

```

LightGBM - LR AUPRC: +0.0170 95% CI [+0.0116, +0.0236] p=0.000
LightGBM - LR AUROC: +0.0125 95% CI [+0.0083, +0.0173] p=0.000

```

```

In [24]: fig, axes = plt.subplots(1, 3, figsize=(7.3, 2.65))
plot_models = [m for m in PRED if m != "Baseline (prevalence)"]

ax = axes[0]
for i, name in enumerate(plot_models):
    fpr, tpr, _ = roc_curve(y_te, PRED[name])
    ax.plot(fpr, tpr, color=SERIES[i], dashes=DASH[i] if DASH[i][0] else (1, 0),
            label=f"{name} ({res.loc[name, 'AUROC']:.3f})")
ax.plot([0, 1], [0, 1], color=C["muted"], lw=1.0, dashes=(2, 2))
ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive rate")
ax.set_title("(a) ROC", loc="left"); ax.legend(loc="lower right", fontsize=6,
        title_fontsize=6.8)

ax = axes[1]
for i, name in enumerate(plot_models):
    pr, rc, _ = precision_recall_curve(y_te, PRED[name])
    ax.plot(rc, pr, color=SERIES[i], dashes=DASH[i] if DASH[i][0] else (1, 0),
            label=f"{name} ({res.loc[name, 'AUPRC']:.3f})")
ax.axhline(y_te.mean(), color=C["muted"], lw=1.0, dashes=(2, 2))
ax.text(0.98, y_te.mean() + 0.006, "no skill", ha="right", fontsize=7, color="red")
ax.set_xlabel("Recall"); ax.set_ylabel("Precision"); ax.set_ylim(0, 0.55)
ax.set_title("(b) Precision-recall", loc="left")
ax.legend(loc="upper right", fontsize=6.8, title="Model (AUPRC)", title_fontsize=6.8)

ax = axes[2]
for i, name in enumerate(plot_models):
    pt, pp = calibration_curve(y_te, PRED[name], n_bins=10, strategy="quantile")
    ax.plot(pp, pt, marker="o", ms=3.4, color=SERIES[i],
            dashes=DASH[i] if DASH[i][0] else (1, 0), label=name)
ax.plot([0, 0.6], [0, 0.6], color=C["muted"], lw=1.0, dashes=(2, 2))
ax.set_xlabel("Predicted risk"); ax.set_ylabel("Observed frequency")
ax.set_title("(c) Calibration", loc="left"); ax.legend(loc="upper left", for
fig.tight_layout(); save(fig, "fig_performance"); plt.close(fig)

```

saved figures/fig_performance.pdf

```

In [25]: # Decision-curve analysis + sensitivity as a function of the alert budget
fig, axes = plt.subplots(1, 2, figsize=(7.0, 2.7))
ax = axes[0]
ths = np.linspace(0.05, 0.45, 60)
for i, name in enumerate(plot_models):
    nb = [net_benefit(y_te, PRED[name], t) for t in ths]
    ax.plot(ths, nb, color=SERIES[i], dashes=DASH[i] if DASH[i][0] else (1, 0),
            label=f"{name} ({res.loc[name, 'AUPRC']:.3f})")
ax.plot(ths, [net_benefit(y_te, np.ones_like(y_te, dtype=float), t) for t in ths],
        color=C["muted"], lw=1.0, dashes=(4, 2), label="Treat all")
ax.axhline(0, color=C["ink2"], lw=0.9)
ax.text(0.42, 0.002, "Treat none", fontsize=7, color=C["ink2"], ha="right")
ax.set_xlabel("Risk threshold $p_t$"); ax.set_ylabel("Net benefit")

```

```

ax.set_ylim(-0.02, 0.06); ax.set_title("(a) Decision curve", loc="left")
ax.legend(fontsize=6.8, loc="upper right")

ax = axes[1]
rates = np.linspace(0.02, 0.5, 40)
for i, name in enumerate(plot_models):
    s = [metrics_at_alert(y_te, PRED[name], r)["sens"] for r in rates]
    ax.plot(rates * 100, np.array(s) * 100, color=SERIES[i],
            dashes=DASH[i] if DASH[i][0] else (1, 0), label=name)
ax.plot(rates * 100, rates * 100, color=C["muted"], lw=1.0, dashes=(2, 2))
ax.text(40, 36, "random", fontsize=7, color=C["ink2"], rotation=32)
ax.set_xlabel("Alert rate: % of discharges flagged"); ax.set_ylabel("Readmission rate")
ax.set_title("(b) Yield at a fixed follow-up budget", loc="left")
ax.legend(fontsize=6.8, loc="lower right")
fig.tight_layout(); save(fig, "fig_utility"); plt.close(fig)

```

saved figures/fig_utility.pdf

8. Does correcting the class imbalance help?

Every model in Section 7 carries an imbalance correction *inside* the learner (`class_weight="balanced"` / `scale_pos_weight`), and every one of them is badly calibrated (ECE $\approx$ 0.34, Brier worse than the constant baseline). This is the expected consequence of re-weighting: the model no longer estimates $P(y=1 \mid x)$ but a tilted version of it. We therefore compare five treatments of the same LightGBM configuration:

1. **None** — train on the data as collected;
2. **Class weights** — the Section 7 setting;
3. **SMOTE** — synthetic minority oversampling of the one-hot design matrix to 1:1;
4. **SMOTE-NC** — the categorical-aware variant, applied before encoding;
5. **Random undersampling** of the majority class to 1:1;
6. **None + isotonic recalibration** on a patient-disjoint calibration split.

```

In [26]: from imblearn.over_sampling import SMOTE, SMOTENC
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.calibration import CalibratedClassifierCV

best_lgbm = {k.replace("clf__", ""): v for k, v in SEARCHES["LightGBM"].best
def lgbm(**kw):
    p = dict(n_estimators=500, random_state=RANDOM_STATE, n_jobs=4, verbose=0)
    p.update(best_lgbm); p.update(kw)
    return LGBMClassifier(**p)

# patient-disjoint calibration split carved out of the training set
gss2 = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=7)
fit_i, cal_i = next(gss2.split(X_tr, y_tr, g_tr))

variants = {}

```

```

variants["None"] = Pipeline([("prep", make_prep(False)), ("clf", lgbm())].fit(X_tr, y_tr)
variants["Class weights"] = MODELS["LightGBM"]
variants["SMOTE (1:1)"] = ImbPipeline([("prep", make_prep(False)),
                                       ("res", SMOTE(random_state=RANDOM_STATE)),
                                       ("clf", lgbm())].fit(X_tr, y_tr)
# SMOTE-NC interpolates within the categorical structure instead of across categories
catmask = [not pd.api.types.is_numeric_dtype(f[c]) for c in f.columns]
variants["SMOTE-NC (1:1)"] = ImbPipeline([("res", SMOTENC(categorical_features=catmask,
                                                           random_state=RANDOM_STATE)),
                                       ("prep", make_prep(False)), ("clf", lgbm())].fit(X_tr, y_tr)
variants["Undersampling (1:1)"] = ImbPipeline([("prep", make_prep(False)),
                                                ("res", RandomUnderSampler(random_state=RANDOM_STATE)),
                                                ("clf", lgbm())].fit(X_tr, y_tr)

from sklearn.frozen import FrozenEstimator
_base = Pipeline([("prep", make_prep(False)), ("clf", lgbm())].fit(X_tr, y_tr)
cal = CalibratedClassifierCV(FrozenEstimator(_base), method="isotonic")
cal.fit(X_tr, y_tr)
variants["None + isotonic"] = cal

imb_rows = []
for name, m in variants.items():
    p = m.predict_proba(X_te)[0, 1]
    r = evaluate(y_te, p, name)
    r["pred"] = p
    imb_rows.append(r)
imb = pd.DataFrame(imb_rows).set_index("model")
print(imb.drop(columns=["pred", "threshold@10%"]).round(4).to_string())
reg("imbalance_results", json.loads(imb.drop(columns=["pred"]).reset_index().to_json(orient="records")))
reg("mean_pred_risk", {k: float(v.mean()) for k, v in zip(imb.index, imb["pred"])})
print("\nmean predicted risk (true prevalence = %.3f):" % y_te.mean())
print({k: round(float(v.mean()), 3) for k, v in zip(imb.index, imb["pred"])})

```

	AUROC	AUPRC	Brier	ECE	sens@10%	ppv@10%	nne@10%
10% model							
None	0.6759	0.2233	0.0953	0.0029	0.2366	0.2666	3.7
515 Class weights	0.6770	0.2228	0.2197	0.3434	0.2349	0.2645	3.7
802 SMOTE (1:1)	0.6755	0.2183	0.0955	0.0086	0.2305	0.2597	3.8
507 SMOTE-NC (1:1)	0.6010	0.1554	0.1420	0.1587	0.1667	0.1878	5.3
247 Undersampling (1:1)	0.6716	0.2143	0.2261	0.3511	0.2252	0.2536	3.9
427 None + isotonic	0.6742	0.2113	0.0954	0.0053	0.2323	0.2617	3.8
210							

```

mean predicted risk (true prevalence = 0.113):
{'None': 0.113, 'Class weights': 0.456, 'SMOTE (1:1)': 0.119, 'SMOTE-NC (1:1)': 0.271, 'Undersampling (1:1)': 0.464, 'None + isotonic': 0.112}

```

```

In [27]: fig, axes = plt.subplots(1, 3, figsize=(7.3, 2.65))
         cols = [C["blue"], C["orange"], C["aqua"], C["violet"], C["ink2"], C["muted"]]
         dsh = [(1, 0), (5, 2), (1.6, 1.6), (7, 2, 1.6, 2), (3, 1, 1, 1), (2, 2)]

```

```

ax = axes[0]
xs = np.arange(len(imb))
ax.bar(xs, imb.AUPRC, color=C["blue"], width=0.6, zorder=3)
ax.axhline(y_te.mean(), color=C["orange"], dashes=(4, 2), lw=1.2, zorder=4)
ax.text(len(imb) - 0.4, y_te.mean() + 0.004, "no skill", ha="right", fontsize=12)
for i, v in enumerate(imb.AUPRC):
    ax.text(i, v + 0.005, f"{v:.3f}", ha="center", fontsize=7)
ax.set_xticks(xs); ax.set_xticklabels(imb.index, rotation=28, ha="right", fontsize=12)
ax.set_ylabel("AUPRC"); ax.set_ylim(0, 0.3); ax.set_title("(a) Ranking is uncalibrated")

ax = axes[1]
ax.bar(xs, imb.ECE, color=C["orange"], width=0.6, zorder=3)
for i, v in enumerate(imb.ECE):
    ax.text(i, v + 0.008, f"{v:.3f}", ha="center", fontsize=7)
ax.set_xticks(xs); ax.set_xticklabels(imb.index, rotation=28, ha="right", fontsize=12)
ax.set_ylabel("Expected calibration error"); ax.set_title("(b) Calibration is poor")

ax = axes[2]
for i, (name, p) in enumerate(zip(imb.index, imb["pred"])):
    pt, pp = calibration_curve(y_te, p, n_bins=10, strategy="quantile")
    ax.plot(pp, pt, marker="o", ms=3.2, color=cols[i], dashes=dsh[i], label=name)
ax.plot([0, 1], [0, 1], color=C["muted"], lw=1.0, dashes=(2, 2))
ax.set_xlabel("Predicted risk"); ax.set_ylabel("Observed frequency")
ax.set_title("(c) Calibration curves", loc="left"); ax.legend(fontsize=6.5, loc="right")
fig.tight_layout(); save(fig, "fig_imbalance"); plt.close(fig)

saved_figures/fig_imbalance.pdf

```

9. How much optimism does encounter-level splitting create?

We now repeat the protocol with the naive split used by most published work on this dataset: a stratified random split of *encounters*, which allows the same patient to appear on both sides. Model configurations are held fixed, so the only thing that changes is the split.

```

In [28]: tr2, te2 = train_test_split(np.arange(len(f)), test_size=0.25, stratify=y,
                                     random_state=RANDOM_STATE)
overlap = len(set(groups[tr2]) & set(groups[te2]))
print(f"patients appearing on both sides of the naive split: {overlap:,} "
      f"({overlap/pd.Series(groups[te2]).nunique()*100:.1f}% of test patient")
reg("naive_overlap_patients", int(overlap))
reg("naive_overlap_pct", float(overlap / pd.Series(groups[te2]).nunique() * 100))

split_rows = []
for name in ["Logistic regression", "Random forest", "XGBoost", "LightGBM"]:
    est = MODELS[name]
    import sklearn
    m2 = sklearn.base.clone(est).fit(f.iloc[tr2], y[tr2])
    p2 = m2.predict_proba(f.iloc[te2])[:, 1]
    split_rows.append(dict(model=name,
                           AUROC_patient=res.loc[name, "AUROC"], AUPRC_patient=res.loc[name, "AUPRC"],
                           AUROC_encounter=roc_auc_score(y[te2], p2),

```

```

AUPRC_encounter=average_precision_score(y[te2], p
split = pd.DataFrame(split_rows).set_index("model")
split["dAUROC"] = split.AUROC_encounter - split.AUROC_patient
split["dAUPRC"] = split.AUPRC_encounter - split.AUPRC_patient
split["rel_AUPRC_%"] = split.dAUPRC / split.AUPRC_patient * 100
print(split.round(4).to_string())
reg("split_results", json.loads(split.reset_index().to_json(orient="records"
reg("max_optimism_auprc_pct", float(split["rel_AUPRC_%"].max()))
reg("mean_optimism_auroc", float(split["dAUROC"].mean()))

```

patients appearing on both sides of the naive split: 7,836 (35.8% of test patients)

	AUROC_patient	AUPRC_patient	AUROC_encounter	AUPRC_en
counter	dAUROC	dAUPRC	rel_AUPRC_%	
model				
Logistic regression	0.6645	0.2057	0.6648	
0.2070 0.0003 0.0013	0.6464			
Random forest	0.6742	0.2187	0.6786	
0.2232 0.0043 0.0045	2.0801			
XGBoost	0.6748	0.2166	0.6776	
0.2191 0.0028 0.0025	1.1620			
LightGBM	0.6770	0.2228	0.6804	
0.2215 0.0034 -0.0012	-0.5605			

Out[28]: 0.002707333446400173

```

In [29]: fig, axes = plt.subplots(1, 2, figsize=(7.0, 2.7))
w = 0.36; xs = np.arange(len(split))
for ax, (a, b, lab) in zip(axes, [("AUROC_patient", "AUROC_encounter", "AURC
("AUPRC_patient", "AUPRC_encounter", "AUPRC
ax.bar(xs - w/2, split[a], width=w, color=C["blue"], label="Patient-level
ax.bar(xs + w/2, split[b], width=w, color=C["orange"], label="Encounter-
for i in xs:
    ax.text(i - w/2, split[a].iloc[i] + 0.004, f"{split[a].iloc[i]:.3f}")
    ax.text(i + w/2, split[b].iloc[i] + 0.004, f"{split[b].iloc[i]:.3f}")
ax.set_xticks(xs); ax.set_xticklabels([m.replace(" ", "\n") for m in spl
ax.set_ylabel(lab); ax.legend(fontsize=7, loc="lower right")
ax.set_ylim(0, max(split[b]) * 1.25)
axes[0].set_title("(a) Discrimination", loc="left")
axes[1].set_title("(b) Ranking of the positive class", loc="left")
fig.tight_layout()
fig.text(0.5, -0.02, f"Encounter-level splitting shares {RESULTS['naive_over
"patients with the training set and inflates every metr
ha="center", fontsize=7.5, color=C["ink2"])
save(fig, "fig_leakage"); plt.close(fig)

```

saved figures/fig_leakage.pdf

A diagnostic worth recording: after training on a perfectly balanced SMOTE sample the mean predicted risk is still ~0.11, whereas undersampling and class weighting push it to ~0.46. Interpolating between one-hot encoded rows produces *fractional* category values that never occur in real data, so the learner can separate synthetic from real minority cases and the oversampling is largely inert. SMOTE-NC, which respects the categorical structure, does shift the

predictions — and degrades discrimination. Neither variant improves on simply leaving the class distribution alone.

10. Where does the signal come from? Feature-block ablation

```
In [30]: blocks = list(FEATURE_BLOCKS)
abl = []
cum = []
for b in blocks:
    cum = cum + FEATURE_BLOCKS[b]
    m = Pipeline([("prep", make_prep(False, cum)), ("clf", lgblm())].fit(X_t
    p = m.predict_proba(X_te)[: , 1]
    abl.append(dict(setting=f"+ {b}", kind="cumulative", n_feat=len(cum),
                    AUROC=roc_auc_score(y_te, p), AUPRC=average_precision_sc
    print(f"cumulative {b:20s} AUPRC={abl[-1]['AUPRC']:.4f}")

full = [c for c in f.columns]
for b in blocks:
    keep = [c for c in full if c not in FEATURE_BLOCKS[b]]
    m = Pipeline([("prep", make_prep(False, keep)), ("clf", lgblm())].fit(X_
    p = m.predict_proba(X_te)[: , 1]
    abl.append(dict(setting=f"- {b}", kind="leave-one-out", n_feat=len(keep)
                    AUROC=roc_auc_score(y_te, p), AUPRC=average_precision_sc
    print(f"drop {b:20s} AUPRC={abl[-1]['AUPRC']:.4f}")
ablation = pd.DataFrame(abl)
full_auprc = float(variants["None"].predict_proba(X_te)[: , 1] @ np.ones(len(
                    average_precision_score(y_te, variants["None"].predict_pr
ablation["delta_vs_full"] = ablation.AUPRC - full_auprc
reg("ablation", json.loads(ablation.to_json(orient="records")))
reg("full_model_auprc", full_auprc)
print(ablation.round(4).to_string(index=False))
```

cumulative	Demographic	AUPRC=0.1167
cumulative	Administrative	AUPRC=0.1672
cumulative	Clinical	AUPRC=0.1725
cumulative	Medication	AUPRC=0.1764
cumulative	Prior utilisation	AUPRC=0.2233
drop	Demographic	AUPRC=0.2231
drop	Administrative	AUPRC=0.2080
drop	Clinical	AUPRC=0.2190
drop	Medication	AUPRC=0.2215
drop	Prior utilisation	AUPRC=0.1764

	setting	kind	n_feat	AUROC	AUPRC	delta_vs_full
	+ Demographic	cumulative	3	0.5104	0.1167	-0.1066
	+ Administrative	cumulative	8	0.6053	0.1672	-0.0561
	+ Clinical	cumulative	22	0.6254	0.1725	-0.0508
	+ Medication	cumulative	27	0.6320	0.1764	-0.0469
+ Prior utilisation		cumulative	34	0.6759	0.2233	0.0000
	- Demographic	leave-one-out	31	0.6762	0.2231	-0.0002
	- Administrative	leave-one-out	29	0.6517	0.2080	-0.0153
	- Clinical	leave-one-out	20	0.6706	0.2190	-0.0043
	- Medication	leave-one-out	29	0.6744	0.2215	-0.0018
- Prior utilisation		leave-one-out	27	0.6320	0.1764	-0.0469

```
In [31]: fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.9))
cu = ablation[ablation.kind == "cumulative"]
ax = axes[0]
ax.plot(range(len(cu)), cu.AUPRC, marker="o", ms=5, color=C["blue"], zorder=
ax.axhline(y_te.mean(), color=C["muted"], dashes=(3, 2), lw=1.0)
ax.text(len(cu) - 1, y_te.mean() + 0.004, "no skill", ha="right", fontsize=7
for i, (v, n) in enumerate(zip(cu.AUPRC, cu.n_feat)):
    ax.annotate(f"{v:.3f}", (i, v), textcoords="offset points", xytext=(0, 7
ax.set_xticks(range(len(cu)))
ax.set_xticklabels([s.replace("+ ", "").replace(" ", "\n") for s in cu.setti
ax.set_ylabel("Test AUPRC"); ax.set_ylim(0.10, 0.25)
ax.set_title("(a) Cumulative feature blocks", loc="left")

lo = ablation[ablation.kind == "leave-one-out"].sort_values("delta_vs_full")
ax = axes[1]
ax.barh([s.replace("- ", "") for s in lo.setting], lo.delta_vs_full, color=C
        height=0.6, zorder=3)
ax.axvline(0, color=C["ink2"], lw=0.9)
ax.set_xlabel("$\Delta$ AUPRC when the block is removed")
ax.set_title("(b) Leave-one-block-out", loc="left")
fig.tight_layout(); save(fig, "fig_ablation"); plt.close(fig)
```

saved figures/fig_ablation.pdf

11. Interpretability

Global attribution for the (uncorrected, well-calibrated) LightGBM model via TreeSHAP, alongside the odds ratios of the regularised logistic regression as a transparent cross-check.

```
In [32]: import shap
best_pipe = variants["None"]
prep_fitted = best_pipe.named_steps["prep"]
feat_names = [n.split("__", 1)[1] for n in prep_fitted.get_feature_names_out
sub = rng.choice(len(X_te), size=3000, replace=False)
Xs = prep_fitted.transform(X_te.iloc[sub])
expl = shap.TreeExplainer(best_pipe.named_steps["clf"])
sv = expl.shap_values(Xs)
if isinstance(sv, list):
    sv = sv[1]
mean_abs = np.abs(sv).mean(0)
order = np.argsort(mean_abs)[::-1][:15]
```

```

reg("top_shap", [[feat_names[i], float(mean_abs[i])] for i in order])

fig = plt.figure(figsize=(6.0, 4.2))
shap.summary_plot(sv[:, order], pd.DataFrame(Xs[:, order], columns=[feat_names[i] for i in order]),
                  show=False, plot_size=None, color_bar_label="Feature value")
f_ = plt.gcf()
f_.set_size_inches(6.0, 4.2)
plt.title("SHAP contributions to predicted 30-day readmission risk", loc="left")
plt.xlabel("SHAP value (log-odds contribution)", fontsize=8.5)
plt.tight_layout(); save(f_, "fig_shap"); plt.close("all")

saved_figures/fig_shap.pdf

```

```

In [33]: lr_pipe = MODELS["Logistic regression"]
lr_names = [n.split("__", 1)[1] for n in lr_pipe.named_steps["prep"].get_feature_names_out()]
coef = lr_pipe.named_steps["clf"].coef_[0]
or_tbl = pd.DataFrame({"feature": lr_names, "coef": coef, "OR": np.exp(coef)})
or_tbl = or_tbl.reindex(or_tbl.coef.abs().sort_values(ascending=False).index)
print(or_tbl.round(3).to_string(index=False))
reg("top_odds_ratios", json.loads(or_tbl.to_json(orient="records")))

```

	feature	coef	OR
	discharge_group_Transfer to another facility	0.322	1.380
	discharge_group_Home	-0.261	0.771
	diag_2_group_Neoplasms	0.253	1.288
	medical_specialty_Nephrology	0.228	1.256
	medical_specialty_ObstetricsandGynecology	-0.210	0.810
	diag_3_group_Missing	-0.197	0.821
	number_inpatient	0.176	1.192
	payer_code_Missing	0.171	1.186
medical_specialty_Surgery-Cardiovascular/Thoracic		-0.170	0.843
medical_specialty_Orthopedics-Reconstructive		-0.164	0.849
	payer_code_0G	0.159	1.172
	diag_3_group_Neoplasms	0.144	1.155
	diag_2_group_Respiratory	-0.138	0.871
	diag_1_group_Respiratory	-0.136	0.872
	diabetesMed_No	-0.133	0.875

```

Out[33]: [{ 'feature': 'discharge_group_Transfer to another facility',
            'coef': 0.3222589549,
            'OR': 1.3802421501},
          { 'feature': 'discharge_group_Home',
            'coef': -0.2605763339,
            'OR': 0.7706073307},
          { 'feature': 'diag_2_group_Neoplasms',
            'coef': 0.2527070935,
            'OR': 1.2875061027},
          { 'feature': 'medical_specialty_Nephrology',
            'coef': 0.2279854816,
            'OR': 1.2560670892},
          { 'feature': 'medical_specialty_ObstetricsandGynecology',
            'coef': -0.2102401964,
            'OR': 0.8103895699},
          { 'feature': 'diag_3_group_Missing',
            'coef': -0.1966335779,
            'OR': 0.8214915908},
          { 'feature': 'number_inpatient', 'coef': 0.1755842571, 'OR': 1.1919424141},
          { 'feature': 'payer_code_Missing', 'coef': 0.170616861, 'OR': 1.186036245
3},
          { 'feature': 'medical_specialty_Surgery-Cardiovascular/Thoracic',
            'coef': -0.1702989598,
            'OR': 0.8434126324},
          { 'feature': 'medical_specialty_Orthopedics-Reconstructive',
            'coef': -0.1635698666,
            'OR': 0.8491071727},
          { 'feature': 'payer_code_OG', 'coef': 0.1588479165, 'OR': 1.172159667},
          { 'feature': 'diag_3_group_Neoplasms',
            'coef': 0.1443640633,
            'OR': 1.1553046359},
          { 'feature': 'diag_2_group_Respiratory',
            'coef': -0.1375429364,
            'OR': 0.8714969303},
          { 'feature': 'diag_1_group_Respiratory',
            'coef': -0.1364925751,
            'OR': 0.8724127978},
          { 'feature': 'diabetesMed_No', 'coef': -0.1332215491, 'OR': 0.8752711551}]

```

12. Export: tables for the manuscript

Every table in the paper is generated here, so the manuscript can never drift from the analysis.

```

In [34]: def latex_table(header, rows, colspec, note=None):
          L = [r"\begin{tabular}{%s}" % colspec, r"\hline\noalign{\smallskip}",
              " & ".join(header) + r" \\", r"\noalign{\smallskip}\hline\noalign{\smallskip}"]
          L += [" & ".join(r) + r" \\ " for r in rows]
          L += [r"\noalign{\smallskip}\hline", r"\end{tabular}"]
          if note:
              L.append(r"\[2pt] {\footnotesize %s}" % note)
          return "\n".join(L)

def esc(s):

```

```

        return str(s).replace("_", r"\_").replace("%", r"\%").replace("&", r"&")

# ---- Table 1: cohort characteristics by outcome ----
rows = []
def add_num(label, col):
    a, b = f.loc[y == 1, col], f.loc[y == 0, col]
    p = stats.mannwhitneyu(a, b)[1]
    rows.append([label,
                  f"{b.median():.0f} [{b.quantile(.25):.0f}--{b.quantile(.75):.0f}]",
                  f"{a.median():.0f} [{a.quantile(.25):.0f}--{a.quantile(.75):.0f}]",
                  p])
def add_cat(label, col, level):
    ind = (f[col] == level)
    ct = pd.crosstab(ind, y)
    p = stats.chi2_contingency(ct)[1]
    rows.append([label, f"{ind[y==0].mean()*100:.1f}%\\%", f"{ind[y==1].mean()*100:.1f}%\\%", p])

add_num("Age (years, median [IQR])", "age_mid")
add_num("Length of stay (days)", "time_in_hospital")
add_num("Laboratory procedures", "num_lab_procedures")
add_num("Medications administered", "num_medications")
add_num("Diagnoses recorded", "number_diagnoses")
add_num("Prior inpatient visits", "number_inpatient")
add_num("Prior emergency visits", "number_emergency")
add_num("Prior outpatient visits", "number_outpatient")
add_cat("Female", "gender", "Female")
add_cat("Race recorded as Caucasian", "race", "Caucasian")
add_cat("Race missing", "race", "Missing")
add_cat("Discharged home", "discharge_group", "Home")
add_cat("Transferred to another facility", "discharge_group", "Transfer to a different facility")
add_cat("HbA1c not measured", "A1C", "Not measured")
add_cat("Primary diagnosis: circulatory", "diag_1_group", "Circulatory")
add_cat("Primary diagnosis: diabetes", "diag_1_group", "Diabetes")
add_cat("On diabetes medication", "diabetesMed", "Yes")
add_cat("Any medication dose change", "change", "Ch")

t1 = latex_table([r"Characteristic", r"Not readmitted <=$30 d", r"Readmitted >=$30 d"],
                  rows, "lccc",
                  note=f"$n={int((y==0).sum()):,}$ versus $n={int((y==1).sum()):,}$, $p$-value: {p:.1f}%\\%")
(TAB / "tab_cohort.tex").write_text(t1)

# ---- Table 2: main model comparison ----
rows = []
for name in res.index:
    r = res.loc[name]
    rows.append([esc(name),
                  f"{r.AUROC:.3f} [{r.AUROC_lo:.3f}, {r.AUROC_hi:.3f}]",
                  f"{r.AUPRC:.3f} [{r.AUPRC_lo:.3f}, {r.AUPRC_hi:.3f}]",
                  f"{r.Brier:.3f}", f"{r.ECE:.3f}",
                  f"{r['sens@10%']*100:.1f}%", f"{r['ppv@10%']*100:.1f}%"])
(TAB / "tab_results.tex").write_text(latex_table(
    ["Model", "AUROC [95\\% CI]", "AUPRC [95\\% CI]", "Brier", "ECE",
     "Sens@10\\%", "PPV@10\\%"], rows, "lcccccc",
    note="Patient-disjoint test set ($n=%d$ encounters, prevalence %.1f\\%)  

    All models except the baseline carry an in-learner imbalance correction factor  

    Sens@10\\% and PPV@10\\% are computed at the threshold that flag")

```

```

        "highest-risk decile." % (len(y_te), y_te.mean() * 100)))

# ---- Table 3: imbalance treatments ----
rows = [[esc(n), f"{r.AUROC:.3f}", f"{r.AUPRC:.3f}", f"{r.Brier:.3f}", f"{r.
        f"{RESULTS['mean_pred_risk'][n]:.3f}", f"{r['sens@10%']*100:.1f}"]
        for n, r in imb.iterrows()]
(TAB / "tab_imbalance.tex").write_text(latex_table(
    ["Imbalance treatment", "AUROC", "AUPRC", "Brier", "ECE", "Mean  $\hat{p}$ 
    rows, "lcccccc",
    note="Identical LightGBM configuration throughout; only the treatment of
        "distribution changes. The observed event rate in the test set is %

# ---- Table 4: splitting strategy ----
rows = [[esc(n), f"{r.AUROC_patient:.3f}", f"{r.AUROC_encounter:.3f}", f"{r.
        f"{r.AUPRC_patient:.3f}", f"{r.AUPRC_encounter:.3f}", f"{r.dAUPRC:+
        for n, r in split.iterrows()]
(TAB / "tab_split.tex").write_text(latex_table(
    ["Model", "AUROC (pat.)", "AUROC (enc.)", "$\\Delta$", "AUPRC (pat.)", "
    rows, "lcccccc",
    note="\\emph{pat.} = patient-disjoint split; \\emph{enc.} = naive random
        "encounters, in which %.0f\\%% of test patients also appear in trai
        % RESULTS["naive_overlap_pct"]]))

# ---- Table 5: ablation ----
rows = [[esc(r.setting), str(r.n_feat), f"{r.AUROC:.3f}", f"{r.AUPRC:.3f}",
        for _, r in ablation.iterrows()]
(TAB / "tab_ablation.tex").write_text(latex_table(
    ["Feature set", "$k$", "AUROC", "AUPRC", "$\\Delta$AUPRC vs.\\ full"], r
    note="Upper block: blocks added cumulatively. Lower block: each block re
        "full model. Uncorrected LightGBM, patient-level split.))

# ---- Table 6: top predictors ----
shap_rank = {n: i + 1 for i, (n, _) in enumerate(RESULTS["top_shap"])}
rows = []
for _, r in or_tbl.iterrows():
    rows.append([esc(r.feature), f"{r.OR:.2f}", str(shap_rank.get(r.feature,
(TAB / "tab_predictors.tex").write_text(latex_table(
    ["Predictor (encoded)", "Odds ratio (LR)", "SHAP rank"], rows, "lcc",
    note="Odds ratios from the  $L_2$ -regularised logistic regression (standa
        "numeric predictors); SHAP rank is the position in the mean  $\\$SHAP$ 
        "of the LightGBM model ('--' = outside the top 15)."))

print("tables written:", sorted(p.name for p in TAB.glob("*.tex")))

```

```

tables written: ['tab_ablation.tex', 'tab_ablation_note.tex', 'tab_ablation_
wrapper.tex', 'tab_cohort.tex', 'tab_cohort_note.tex', 'tab_cohort_wrapper.t
ex', 'tab_extraction.tex', 'tab_extraction_wrapper.tex', 'tab_imbalance.te
x', 'tab_imbalance_note.tex', 'tab_imbalance_wrapper.tex', 'tab_missing.te
x', 'tab_missing_wrapper.tex', 'tab_predictors.tex', 'tab_predictors_note.te
x', 'tab_predictors_wrapper.tex', 'tab_results.tex', 'tab_results_note.tex',
'tab_results_wrapper.tex', 'tab_screening.tex', 'tab_search_log.tex', 'tab_s
plit.tex', 'tab_split_note.tex', 'tab_split_wrapper.tex']

```

```

In [35]: RESULTS["runtime_minutes"] = round((time.time() - t0) / 60, 1)
RESULTS["versions"] = {m.__name__: getattr(m, "__version__", "?")
                        for m in [np, pd, mpl]}

```

```
(ROOT / "artifacts" / "results.json").write_text(json.dumps(RESULTS, indent=
print(f"\nAll results registered -> artifacts/results.json  "
      f"({len(RESULTS)} keys, {RESULTS['runtime_minutes']} min runtime)")
```

All results registered -> artifacts/results.json (55 keys, 8.0 min runtime)